

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:

```
N_STEPS_DEFAULT = 1000
EPSILON_DEFAULT = 0.1
MARGIN_DEFAULT = 1e-6
```

1. Статистический анализ результатов прямого численного моделирования

(a) Определим процедуры обновления состояния `step_stochastic_em`, `step_stochastic_rk4`, задающие временной шаг стохастической системы, и запишем стохастические системы FHN и SN. В работе будем использовать стохастический метод Рунге-Кутты 4-го порядка для обновления состояния (процедура `step_stochastic_rk4`).

In [3]:

```
def step_stochastic_em(
    model_func: callable,
    params: dict,
    state: np.ndarray,
    dt: float,
    epsilon: float,
    noise_mask: np.ndarray = np.array([1, 1]),
    rng: np.random.Generator = np.random.default_rng(),
):
    """
    Euler-Maruyama step for stochastic system given by `model_func` with `params`.

    Args:
        dt: integration step
        epsilon: noise intensity (when set to 0, the system is considered deterministic)
        noise_mask: denotes the equations to which noise should be added (noise vector i

    Returns:
        system state after the randomized step
    """
    drift = model_func(0, state, **params)

    dW = rng.normal(0., 1., size=2) * np.sqrt(dt)
    noise = epsilon * dW * noise_mask

    return state + drift * dt + noise


def step_stochastic_rk4(
    model_func: callable,
    params: dict,
    state: np.ndarray,
    dt: float,
    epsilon: float,
    noise_mask: np.ndarray,
```

```

rng: np.random.Generator,
):
    """
    Runge-Kutta 4-th order step for stochastic system given by `model_func` with `params`

    First, the deterministic RK4 step is performed, and then the computed state is augmented with noise.

    Args:
        dt: integration step
        epsilon: noise intensity (when set to 0, the system is considered deterministic)
        noise_mask: denotes the equations to which noise should be added (noise vector i

    Returns:
        system state after the randomized step
    """
    k1 = dt * model_func(0, state, **params)
    k2 = dt * model_func(0, state + k1/2, **params)
    k3 = dt * model_func(0, state + k2/2, **params)
    k4 = dt * model_func(0, state + k3, **params)

    state_rk = state + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    dW = rng.normal(0., 1., size=2) * np.sqrt(dt)
    noise = epsilon * dW * noise_mask

    return state_rk + noise

```

FHN с аддитивным шумом в 1-м уравнении:

In [4]:

```

def model_FHN(
    time: float,
    state: np.ndarray,
    delta: float = 0.1,
    a: float = 1,
):
    """
    FitzHugh-Nagumo neural model.
    Accepts `time` argument for interface consistency, but does not use it.

    Parameter restrictions:
    - delta << 1 (delta = 0.1)
    - a > 0
    """
    x, y = state
    return np.array([
        (x - (x ** 3) / 3 - y) / delta,
        x + a
    ])

```

SN с аддитивным шумом в обоих уравнениях:

In [5]:

```

def model_SN(
    time: float,
    state: np.ndarray,
    a: float = 1,
    b: float = 2,

```

```

):
    """
    Saltzman-Nicolis climate model.
    Accepts `time` argument for interface consistency, but does not use it.

    Parameter restrictions:
    - a > 0
    - b = 2
    """
    x, y = state
    return np.array([
        y - x,
        -a * x + b * y - (x ** 2) * y
    ])

```

(б) Для устойчивых равновесий построим облака случайных траекторий для разных значений интенсивности шума .

In [6]:

```

def simulate_trajectory(
    model_func: callable,
    params: dict,
    state_init: np.ndarray,
    time_span: np.ndarray,
    epsilon: float,
    noise_mask: np.ndarray,
    rng: np.random.Generator,
    method: str = 'rk4', # 'em' for Euler-Maruyama step
):
    n = len(time_span)
    dt = time_span[1] - time_span[0]

    trajectory = np.zeros((n, 2))
    trajectory[0] = state_init

    if method == 'rk4':
        step_func = step_stochastic_rk4
    elif method == 'em':
        step_func = step_stochastic_em
    else:
        raise ValueError(f'Unknown step method: {method}!')

    for i in range(n - 1):
        trajectory[i + 1] = step_func(
            model_func=model_func,
            params=params,
            state=trajectory[i],
            dt=dt,
            epsilon=epsilon,
            noise_mask=noise_mask,
            rng=rng,
        )

    return trajectory

def simulate_stochastic_cloud(
    ax: plt.Axes,

```

```

model_func: callable,
params: dict,
equilibrium: np.ndarray,
time_span: np.ndarray,
epsilon: float,
noise_mask: np.ndarray,
n_trajectories: int = 1,
spread_init: float = 0.05,
):
    rng = np.random.default_rng()
    trajectories = []

    for _ in range(n_trajectories):
        # start near the equilibrium
        state_init = equilibrium + rng.normal(0, spread_init, size=2)

        trajectory = simulate_trajectory(
            model_func=model_func,
            params=params,
            state_init=state_init,
            time_span=time_span,
            epsilon=epsilon,
            noise_mask=noise_mask,
            rng=rng,
        )
        trajectories.append(trajectory)

        if ax is not None:
            ax.plot(trajectory[:, 0], trajectory[:, 1], alpha=0.3)

    if ax is not None:
        ax.scatter(*equilibrium, color='red', s=50, zorder=100, label='equilibrium')
        ax.set_title(f"Noise cloud ({epsilon=})")
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.legend(loc="upper right")
        ax.grid(True)

    return trajectories

```

Облака траекторий системы FHN вблизи устойчивого равновесия при :

In [7]:

```

delta, a = 0.1, 1.2

equilibrium_expr = np.array([-a, a**3 / 3 - a])

noise_mask = np.array([1, 0]) # add noise to 1st equation only
epsilon_values = [0.01, 0.05, 0.1]

time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

fig, axes = plt.subplots(1, len(epsilon_values), figsize=(18, 4))

for ax, epsilon in zip(axes, epsilon_values):
    simulate_stochastic_cloud(
        ax=ax,
        model_func=model_FHN,

```

```

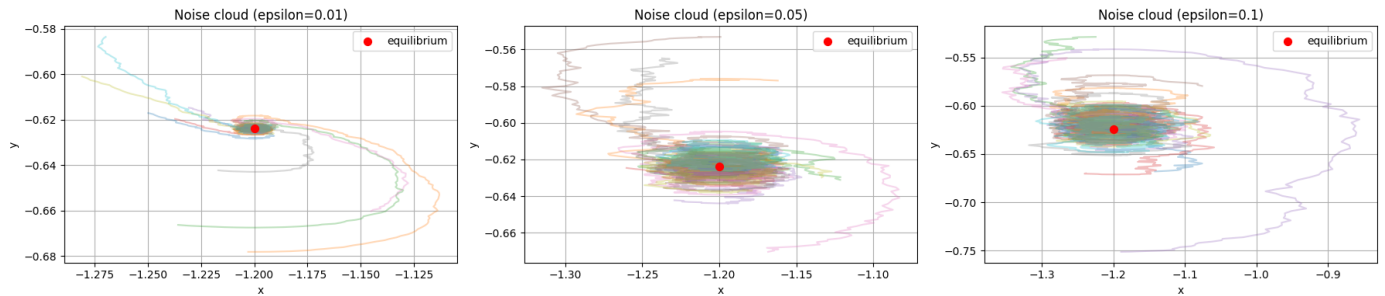
params=dict(delta=delta, a=a),
equilibrium=equilibrium_expr,
time_span=time_span,
epsilon=epsilon,
noise_mask=noise_mask,
n_trajectories=10,
)

```

```

plt.tight_layout()
plt.show()

```



Облака траекторий системы SN вблизи устойчивого равновесия :

In [8]:

```

a, b = 0.75, 2

equilibrium_expr = np.array([np.sqrt(b - a), np.sqrt(b - a)])

noise_mask = np.array([1, 1]) # add noise to both equations
epsilon_values = [0.01, 0.05, 0.1]

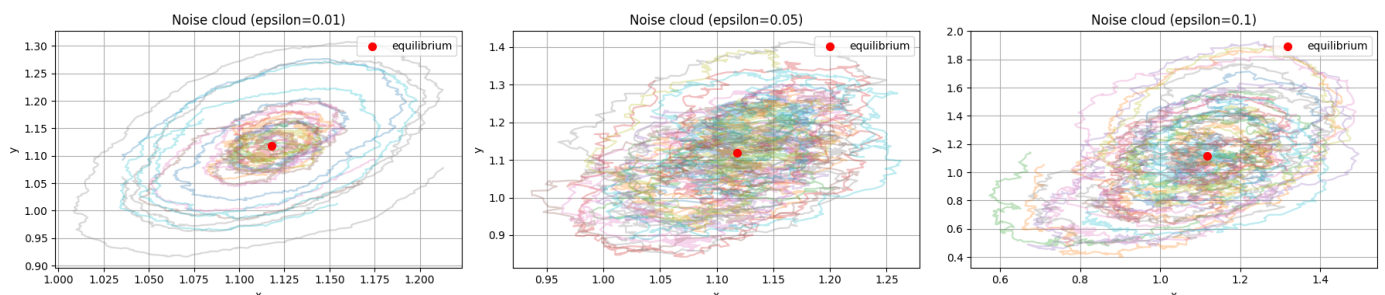
time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

fig, axes = plt.subplots(1, len(epsilon_values), figsize=(18, 4))

for ax, epsilon in zip(axes, epsilon_values):
    simulate_stochastic_cloud(
        ax=ax,
        model_func=model_SN,
        params=dict(a=a, b=b),
        equilibrium=equilibrium_expr,
        time_span=time_span,
        epsilon=epsilon,
        noise_mask=noise_mask,
        n_trajectories=10,
    )

plt.tight_layout()
plt.show()

```



(в) Запишем матрицу ковариации для облака при фиксированном , найдем ее собственные числа .

In [9]:

```
def compute_covariance_matrix_2d(
    trajectories: list[np.ndarray],
    clip_ratio: float = 0.5,
) -> np.ndarray:
    """
    Compute 2x2 covariance matrix for a stochastic cloud.

    Args:
        trajectories: list of trajectories of shape `(n_steps, 2)`

    Returns:
        cov_matrix: covariance matrix
        points_concat: stochastic cloud after clipping the transient
    """
    points = []

    for trajectory in trajectories:
        # clip transient (consider only the asymptotic behavior of the system)
        clip = int(len(trajectory) * clip_ratio)
        traj_stationary = trajectory[clip:]
        points.append(traj_stationary)

    points_concat = np.vstack(points) # shape (N_total, 2)
    cov_matrix = np.cov(points_concat.T)

    return cov_matrix
```

Построим графики зависимостей в зоне устойчивых равновесий . Будем использовать значение .

Будем использовать от до симуляций и шагов по времени в каждой из них, таким образом получим графики для выборок размеров и .

In [10]:

```
def plot_eigenvalues_per_param_value(
    ax: plt.Axes,
    model_func: callable,
    params: dict,
    equilibrium_expr: callable,
    time_span: np.ndarray,
    param_name: str,
    param_values: np.ndarray,
    stable_range: tuple[float, float],
    epsilon: float = 0.1,
    noise_mask: np.ndarray = np.array([1, 1]),
    covariance: bool = True,
    n_trajectories: int = 1,
):
    """
    Plot eigenvalues of the covariance matrix (or analytically derived stochastic sensitivities)
    for different values `param_values` of a given parameter `param_name`.

    Args:
        equilibrium_expr: function, which returns equilibrium point at given model param
        stable_range: parameter range, where given equilibrium remains stable
    """
```

```

epsilon: noise intensity (when set to 0, the system is considered deterministic)
noise_mask: denotes the equations to which noise should be added (noise vector i
covariance: whether to use covariance matrix on a simulation instead of analytic
n_trajectories: number of simulations (unused if `covariance` is False, i.e. ana
"""
lambda1_values = []
lambda2_values = []

for val in param_values:
    params[param_name] = val
    equilibrium = equilibrium_expr(val)

    if covariance:
        trajectories = simulate_stochastic_cloud(
            ax=None,
            model_func=model_func,
            params=params,
            equilibrium=equilibrium,
            time_span=time_span,
            epsilon=epsilon,
            noise_mask=noise_mask,
            n_trajectories=n_trajectories
        )
        matrix = compute_covariance_matrix_2d(trajectories)
        eigenvalues = np.linalg.eigvals(matrix)
    else:
        matrix = stochastic_sensitivity_matrix_2d( # TODO: defined later in the not
            model_func=model_func,
            params=params,
            equilibrium=equilibrium,
            noise_mask=noise_mask,
        )
        eigenvalues = np.linalg.eigvals(matrix) * epsilon ** 2

    lambda1_values.append(eigenvalues[0])
    lambda2_values.append(eigenvalues[1])

ax.axvspan(*stable_range, color='green', alpha=0.3, label='stable equilibria')
ax.plot(param_values, lambda1_values, label=rf'$\lambda_1({param_name})$')
ax.plot(param_values, lambda2_values, label=rf'$\lambda_2({param_name})$')

ax.set_xlabel(param_name)
ax.set_ylabel('eigenvalues')
ax.legend(loc='upper right')
ax.grid(True)

```

В модели FHN при значении параметра равновесие устойчиво.

In [11]:

```

delta, a = 0.1, None # `a` is a variable parameter

noise_mask = np.array([1, 0]) # add noise to 1st equation only

time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

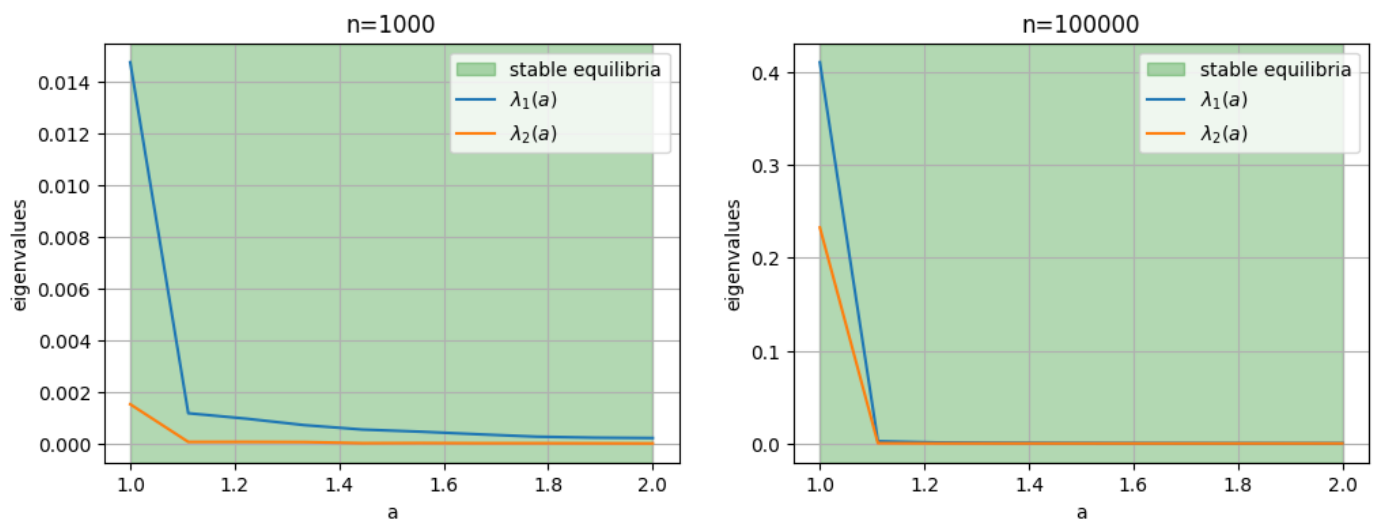
param_name = 'a'
param_values = np.linspace(1 + MARGIN_DEFAULT, 2, num=10)

```

```
def equilibrium_expr(a):
    return np.array([-a, a**3 / 3 - a])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, n_trajectories in zip(axes, [1, 100]):
    plot_eigenvalues_per_param_value(
        ax=ax,
        model_func=model_FHN,
        params=dict(delta=delta, a=a),
        equilibrium_expr=equilibrium_expr,
        time_span=time_span,
        param_name=param_name,
        param_values=param_values,
        stable_range=(param_values[0], param_values[-1]),
        epsilon=EPSILON_DEFAULT,
        noise_mask=noise_mask,
        covariance=True,
        n_trajectories=n_trajectories
    )
    ax.set_title(f'n={n_trajectories * N_STEPS_DEFAULT}')
```



В модели SN при значении параметра устойчивыми являются равновесия:

Вблизи них система ведет себя одинаково с точностью до симметрии. Рассмотрим одно из них, например .

In [12]:

```
a, b = None, 2    # `a` is a variable parameter

noise_mask = np.array([1, 1])    # add noise to both equations

param_name = 'a'
param_values = np.linspace(0, 1 - MARGIN_DEFAULT, num=10)

time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

def equilibrium_expr(a):
    return np.array([np.sqrt(b - a), np.sqrt(b - a)])

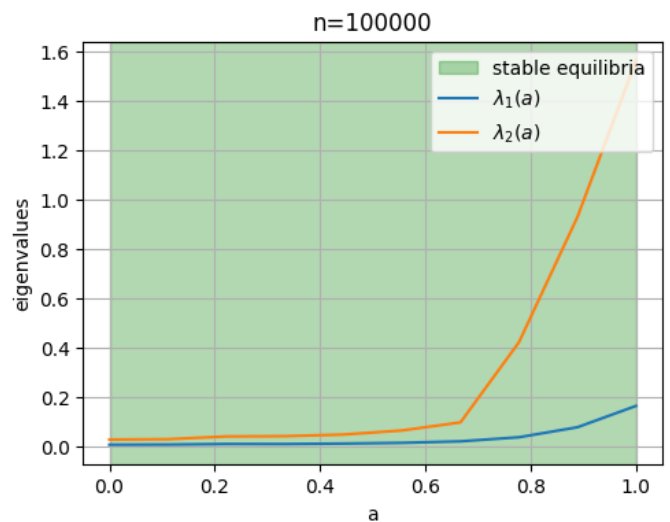
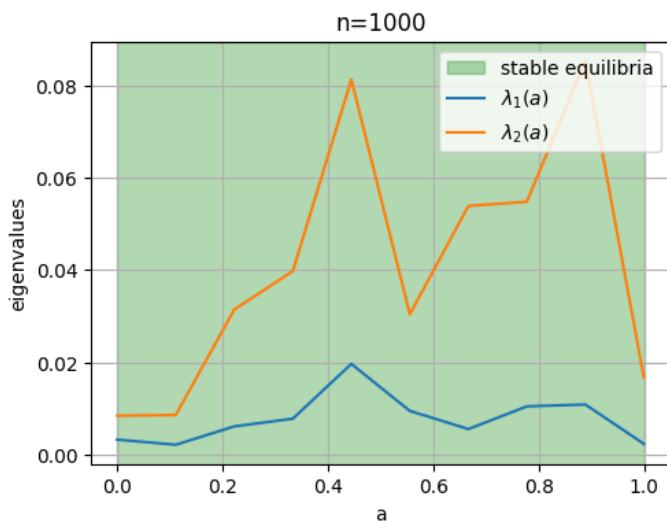
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```



```

for ax, n_trajectories in zip(axes, [1, 100]):
    plot_eigenvalues_per_param_value(
        ax=ax,
        model_func=model_SN,
        params=dict(a=a, b=b),
        equilibrium_expr=equilibrium_expr,
        time_span=time_span,
        param_name=param_name,
        param_values=param_values,
        stable_range=(param_values[0], param_values[-1]),
        epsilon=EPSILON_DEFAULT,
        noise_mask=noise_mask,
        covariance=True,
        n_trajectories=n_trajectories,
    )
    ax.set_title(f'n={n_trajectories * N_STEPS_DEFAULT}')

```



2. Анализ с помощью метода стохастической чувствительности

(а) Запишем решение уравнения Ляпунова в общем виде.

С учетом изначальной формы СДУ:

Имеем

Тогда уравнение (3) имеет вид:

В силу симметрии матриц в левой и правой частях, получаем систему из 3-х уравнений:

Решение относительно :

Получено явное решение, но для удобства программного решения можно представить полученную систему уравнений в матричной форме:

In [13]:

```

def find_jacobian(
    func: callable,

```

```

t: float,
state: np.ndarray,
params: dict,
eps: float = 1e-6,
) -> np.ndarray:
    n = len(state)
    jacobian = np.zeros((n, n))

    for i in range(n):
        perturb = np.zeros(n)
        perturb[i] = eps

        f1 = func(t, state + perturb, **params)
        f2 = func(t, state - perturb, **params)
        jacobian[:, i] = (f1 - f2) / (2 * eps)

    return jacobian

def solve_lyapunov_symmetric_2d(F: np.ndarray, Q: np.ndarray):
    """
    Solve Lyapunov equation  $F W + W F^T + Q = 0$  for 2D symmetric matrix W (stochastic
    and noise matrix Q ( $Q = S S^T$ )).

    The equation is represented in a matrix form with respect to variables w_1, w_2, w_3

    Notes:
        F = [[f'_x(x, y), f'_y(x, y)],
             [g'_x(x, y), g'_y(x, y)]]

        W = [[w_1, w_2],
             [w_2, w_3]]
    """
    a, b = F[0, 0], F[0, 1]
    c, d = F[1, 0], F[1, 1]

    q11 = Q[0, 0]
    q12 = Q[0, 1]
    q22 = Q[1, 1]

    A = np.array([
        [2*a, 2*b, 0],
        [c, a+d, b],
        [0, 2*c, 2*d],
    ], dtype=float)

    rhs = -np.array([q11, q12, q22], dtype=float)

    w1, w2, w3 = np.linalg.solve(A, rhs)

    W = np.array([
        [w1, w2],
        [w2, w3]
    ])

    return W

def stochastic_sensitivity_matrix_2d(

```

```

model_func: callable,
params: dict,
equilibrium: np.ndarray,
noise_mask: np.ndarray = np.array([1, 1]),
):
    """
    Find stochastic sensitivity matrix `W` near the `equilibrium` point for a dynamic sy
    """
    F = find_jacobian(func=model_func, t=0, state=equilibrium, params=params)
    eigenvalues = np.linalg.eigvals(F)

    if np.any(np.real(eigenvalues) >= 0):
        raise ValueError(f"Equilibrium is not asymptotically stable at {params=}. Eigenv

    S = np.array([
        [noise_mask[0], 0.],
        [0., noise_mask[1]]
    ])
    Q = S @ S.T
    W = solve_lyapunov_symmetric_2d(F, Q)

    return W

```

Запишем матрицу стохастической чувствительности для FHN, где

In [14]:

```

delta, a = 0.1, 2
noise_mask = np.array([1, 0])
equilibrium_expr = np.array([-a, a**3 / 3 - a])
W = stochastic_sensitivity_matrix_2d(
    model_func=model_FHN,
    params=dict(delta=delta, a=a),
    equilibrium=equilibrium_expr,
    noise_mask=noise_mask,
)

W

```

Out[14]:

```

array([[ 0.01666667, -0.          ],
       [-0.          ,  0.00166667]])

```

Запишем матрицу стохастической чувствительности для SN, где

In [15]:

```

a, b = 0.5, 2
noise_mask = np.array([1, 1])
equilibrium_expr = np.array([np.sqrt(b - a), np.sqrt(b - a)])
W = stochastic_sensitivity_matrix_2d(
    model_func=model_SN,
    params=dict(a=a, b=b),
    equilibrium=equilibrium_expr,
    noise_mask=noise_mask,
)

W

```

Out[15]:

```
array([[1.41666667, 0.91666667],
       [0.91666667, 5.41666667]])
```

(б) Найдем собственные числа матрицы стохастической чувствительности .

Построим зависимость для из пункта 1 (б) для системы FHN.

In [16]:

```
delta, a = 0.1, None    # `a` is a variable parameter

noise_mask = np.array([1, 0])    # add noise to 1st equation only

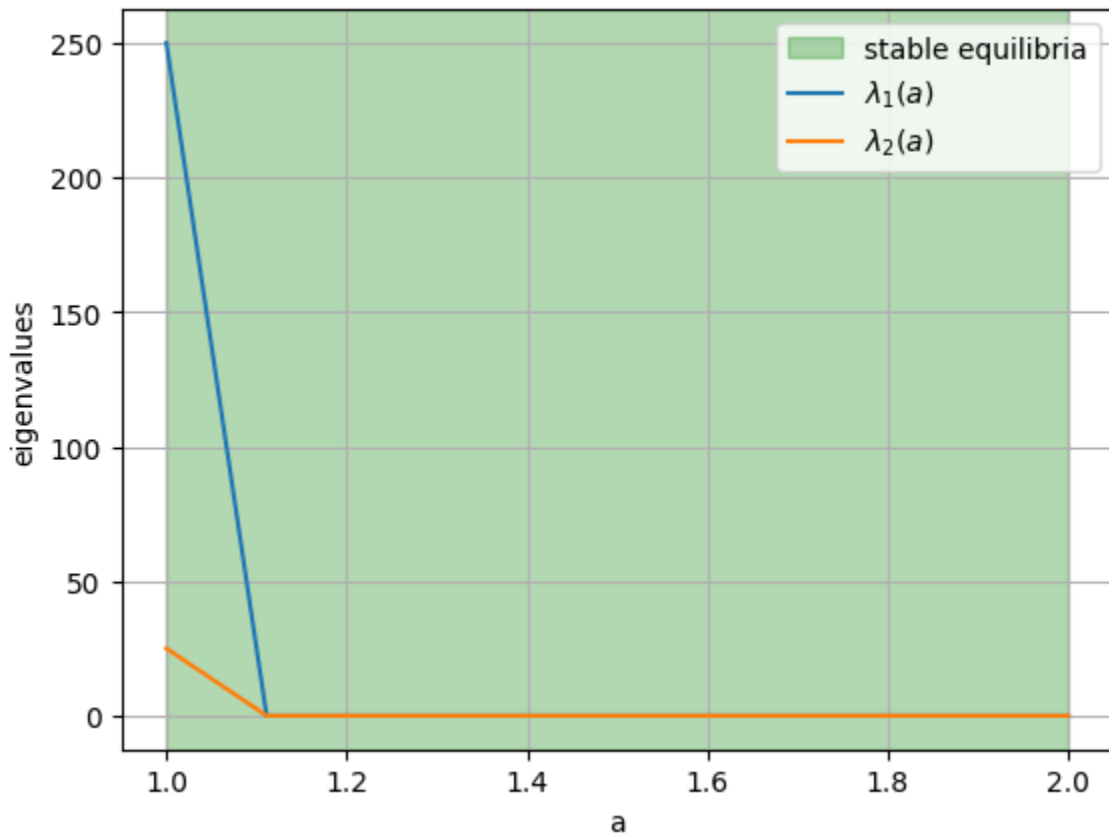
param_name = 'a'
param_values = np.linspace(1 + MARGIN_DEFAULT, 2, num=10)

time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

def equilibrium_expr(a):
    return np.array([-a, a**3 / 3 - a])

fig, ax = plt.subplots()

plot_eigenvalues_per_param_value(
    ax=ax,
    model_func=model_FHN,
    params=dict(delta=delta, a=a),
    equilibrium_expr=equilibrium_expr,
    time_span=time_span,
    param_name=param_name,
    param_values=param_values,
    stable_range=(param_values[0], param_values[-1]),
    epsilon=EPSILON_DEFAULT,
    noise_mask=noise_mask,
    covariance=False,
)
```



Построим зависимость для из пункта 1 (б) для системы SN.

In [17]:

```
a, b = None, 2    # 'a' is a variable parameter

noise_mask = np.array([1, 1])    # add noise to both equations

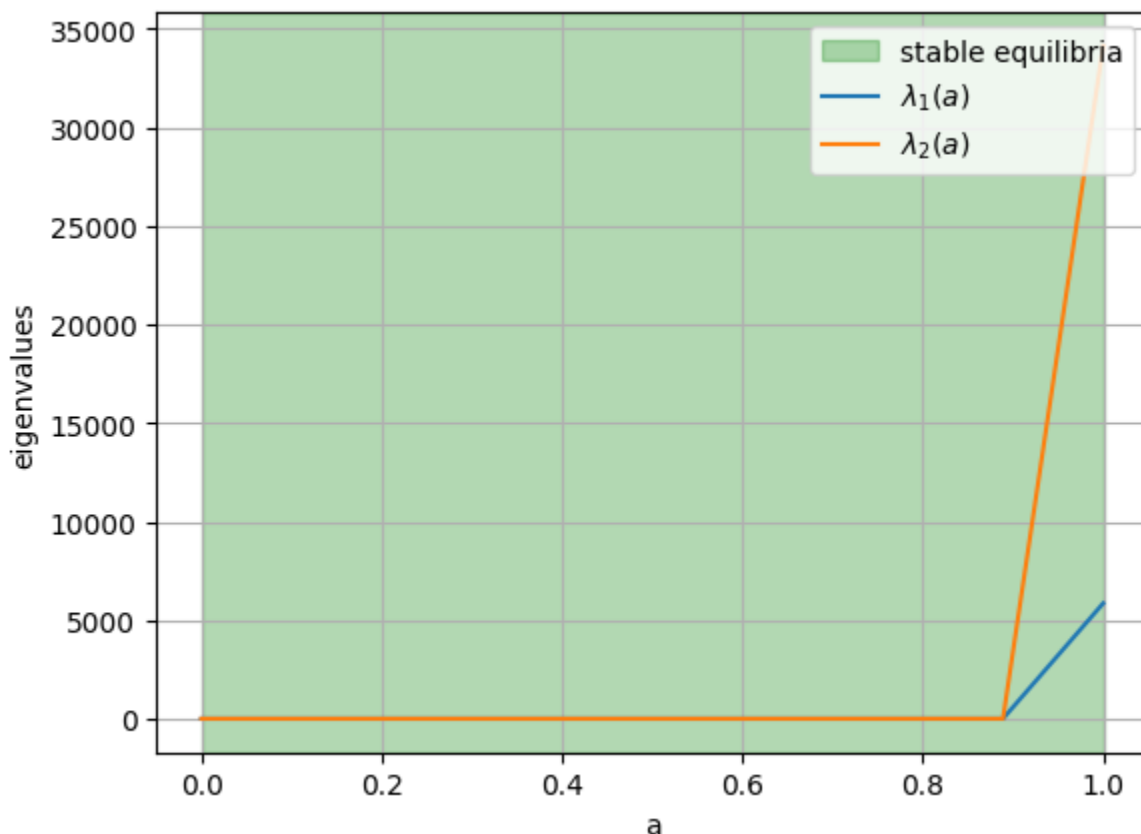
param_name = 'a'
param_values = np.linspace(0, 1 - MARGIN_DEFAULT, num=10)

time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

def equilibrium_expr(a):
    return np.array([np.sqrt(b - a), np.sqrt(b - a)])

fig, ax = plt.subplots()

plot_eigenvalues_per_param_value(
    ax=ax,
    model_func=model_SN,
    params=dict(a=a, b=b),
    equilibrium_expr=equilibrium_expr,
    time_span=time_span,
    param_name=param_name,
    param_values=param_values,
    stable_range=(param_values[0], param_values[-1]),
    epsilon=EPSILON_DEFAULT,
    noise_mask=noise_mask,
    covariance=False,
)
```



Полученные графики схожи с графиками из пункта 1 (в), построенными эмпирически с помощью симуляции, по характеру изменения собственных значений вблизи границ интервала стабильности равновесных точек (зеленая зона). Однако при вычислении аналитическим путем графики получаются менее плавные, и более резко начинают возрастать на границах интервала стабильности.

Доверительные эллипсы

(в) Для выбранных нескольких значений параметра построим доверительные эллипсы (возьмем облако из пункта 1 (б)).

В двумерном случае матрица стохастической чувствительности характеризует дисперсию решений стохастической системы вокруг устойчивого равновесия в форме доверительного эллипса:

Пусть λ -- собственные числа матрицы, V -- матрица собственных векторов, v , записанных по столбцам:

Координаты доверительного эллипса в базисе собственных векторов могут быть записаны так:

где λ , v .

Коэффициент связан с доверительной вероятностью попадания решений стохастической системы в доверительный эллипс:

In [18]:

```

def plot_confidence_ellipse_2d(
    ax: plt.Axes,
    equilibrium: np.ndarray,
    W: np.ndarray,
    epsilon: float,
    confidence: float = 0.95,
    n_points: int = 100,
    major_axes: bool = False,
):
    """
    Plot confidence ellipse for a 2D stochastic system based on the
    stochastic sensitivity matrix W.

    Vectorized and numerically correct version.

    Args:
        ax: Matplotlib axes to plot on
        equilibrium : equilibrium point (x, y) of the deterministic system
        W: 2x2 stochastic sensitivity matrix
        epsilon: noise intensity
        confidence: confidence probability P ( $0 < P < 1$ )
        n_points: number of points to generate the ellipse using the parameterized space
    """
    eigenvalues, eigenvectors = np.linalg.eigh(W)

    idx_sorted = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[idx_sorted]
    eigenvectors = eigenvectors[:, idx_sorted]

    lambda1, lambda2 = eigenvalues

    if lambda1 <= 0 or lambda2 <= 0:
        raise ValueError("W must be positive definite")

    phi = np.linspace(0, 2 * np.pi, n_points)
    circle = np.vstack((np.cos(phi), np.sin(phi)))

    k = np.sqrt(-np.log(1 - confidence))
    axis_scales = np.array([
        np.sqrt(2 * lambda1),
        np.sqrt(2 * lambda2)
    ]) * epsilon * k

    # Translate ellipse points into a basis formed by the eigenvectors of W
    ellipse_local = axis_scales[:, None] * circle
    ellipse = eigenvectors @ ellipse_local
    ellipse += equilibrium.reshape(2, 1)

    ax.plot(ellipse[0], ellipse[1], 'b--', label='confidence ellipse')

    if major_axes:
        for i in range(2):
            axis_length = axis_scales[i]
            v = eigenvectors[:, i]

            p1 = equilibrium - axis_length * v
            p2 = equilibrium + axis_length * v

            ax.plot([p1[0], p2[0]], [p1[1], p2[1]], 'c--')

```

```

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend(loc='upper right')
ax.grid(True)

```

Построим доверительные эллипсы для системы FHN. Рассмотрим несколько значений параметра внутри интервала, где рассматриваемое равновесие сохраняет устойчивость.

In [23]:

```

delta, a = 0.1, 1.2
noise_mask = np.array([1, 0])

param_name = 'a'
param_values = [1.1, 1.5, 1.9]

confidence = 0.95
time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

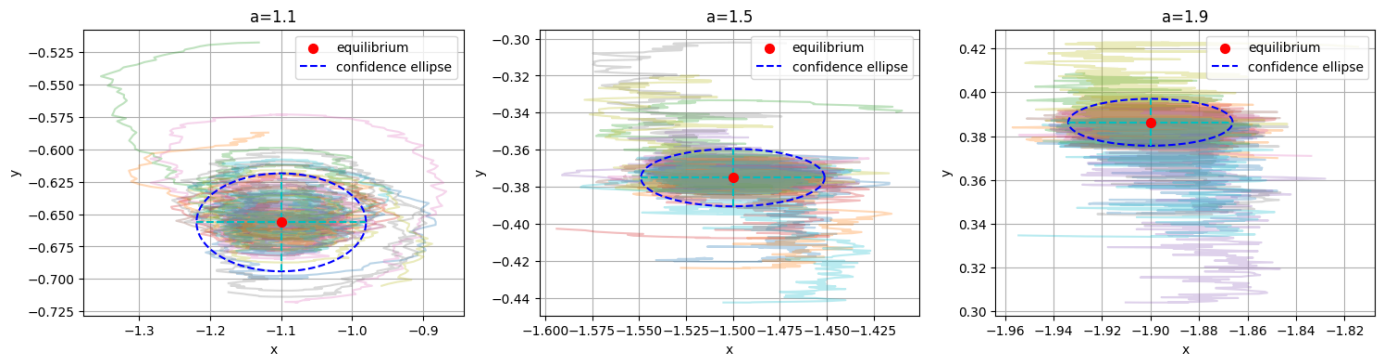
fig, axes = plt.subplots(1, len(param_values), figsize=(18, 4))

for ax, val in zip(axes, param_values):
    a = val
    equilibrium_expr = np.array([-a, a**3 / 3 - a])
    W = stochastic_sensitivity_matrix_2d(
        model_func=model_FHN,
        params=dict(delta=delta, a=a),
        equilibrium=equilibrium_expr,
        noise_mask=noise_mask,
    )

    simulate_stochastic_cloud(
        ax=ax,
        model_func=model_FHN,
        params=dict(delta=delta, a=a),
        equilibrium=equilibrium_expr,
        time_span=time_span,
        epsilon=EPSILON_DEFAULT,
        noise_mask=noise_mask,
        n_trajectories=10,
    )

    plot_confidence_ellipse_2d(
        ax=ax,
        equilibrium=equilibrium_expr,
        W=W,
        epsilon=EPSILON_DEFAULT,
        confidence=confidence,
        major_axes=True,
    )
    ax.set_title(f'{a=}')

```

Построим доверительные эллипсы для системы SN. Рассмотрим несколько значений параметра внутри интервала, где рассматриваемое равновесие сохраняет устойчивость.

In [30]:

```
a, b = 0.75, 2
noise_mask = np.array([1, 1]) # add noise to both equations

param_name = 'a'
param_values = [0.1, 0.5, 0.9]

confidence = 0.95
time_span = np.linspace(0, 10, N_STEPS_DEFAULT)

fig, axes = plt.subplots(1, len(param_values), figsize=(18, 4))

for ax, val in zip(axes, param_values):
    a = val
    equilibrium_expr = np.array([np.sqrt(b - a), np.sqrt(b - a)])
    W = stochastic_sensitivity_matrix_2d(
        model_func=model_SN,
        params=dict(a=a, b=b),
        equilibrium=equilibrium_expr,
        noise_mask=noise_mask,
    )

    simulate_stochastic_cloud(
        ax=ax,
        model_func=model_SN,
        params=dict(a=a, b=b),
        equilibrium=equilibrium_expr,
        time_span=time_span,
        epsilon=EPSILON_DEFAULT,
        noise_mask=noise_mask,
        n_trajectories=10,
    )

    plot_confidence_ellipse_2d(
        ax=ax,
        equilibrium=equilibrium_expr,
        W=W,
        epsilon=EPSILON_DEFAULT,
        confidence=confidence,
        major_axes=True,
    )
    ax.set_title(f'{a=}')
```

